

Inversion attack on FaceNet

Asaf Solomiak, Itay Gross

[github](#)

Introduction

Google's FaceNet is a model that encodes facial images into low-dimensional embedding vectors. These embeddings are widely assumed to be irreversible and therefore safe to store and share without compromising the user's original identity. However, with the rise of generative AI, a critical security question arises: Is the training data truly safe, or can these embeddings be inverted to recover the original face?

Goal

This project investigates the vulnerability of face recognition models to **Model Inversion Attacks**. Our goal is to reconstruct a recognizable face image solely from its FaceNet embedding, effectively inverting the black-box model.

Our research followed 2 general methods:

1. Naive Reconstruction: We first attempted to recover images via direct gradient descent on pixel values.
2. Generative Model Inversion (GMI): Then we used a StyleGAN network as a generative prior.

For our target images, we utilized the CelebA-HQ dataset, a high-quality version of the massive CelebA celebrity face dataset.

Optimization Loop

Each inversion attack attempt, includes an optimization loop in which we feed an image to the FaceNet model, gets an embedding vector, and backpropagates through the FaceNet model into our input image while the model weights are frozen. In this loop, we will use the following losses for the gradient descent:

- Embedding Loss - Cosine Similarity distance or MSE between the embedding of our reconstructed image, to the target image's embedding.
- Total Variation Loss - Mitigates noisy reconstructions by penalizing high differences between adjacent pixels.
- Perceptual Loss - Using a network that measures perceptual distance between our reconstructed image and a general face image.

Evaluation

Inversion attack is tricky to evaluate. We don't really have training and validation sets as we don't train any network. So our evaluation is based on comparison between our reconstructed image and the target image.

At first, we used the following common metrics:

- LPIPS (Learned Perceptual Image Patch Similarity) - similar to the perceptual loss, uses a network to measure perceptual distance, but with the target image.
- SSIM (Structural Similarity Index) - measures the similarity of structural information like brightness, contrast, structure to determine if shapes and textures match.
- PSNR (Peak Signal-to-Noise Ratio) - a mathematical ratio of the maximum pixel power to the noise power.

After a while, we realized that these metrics are sensitive to image features that aren't preserved in the compression of FaceNet, such as background details and shirt color.

We decided to use the ArcFace which is a ResNet model that was trained on face datasets, and was trained to learn identity.

Note on the difference between ArcFace and the LPIPS metric we used: while both of them use a deep neural network to measure perceptual distance, the LPIPS metric uses vgg16 network which was trained on ImageNet to identify objects. It penalizes differences in lighting and background (which FaceNet discards) leading to poor scores even for good inversions. ArcFace, like FaceNet, is invariant to these details and focuses purely on identity, making it the superior metric for this specific task.

First Method - The Classic Approach

The most direct method for model inversion is applying gradient descent in pixel space. We freeze the FaceNet model and initialize a random noise image x . We then treat the pixel values of x as trainable parameters. By calculating the gradient of the embedding error with respect to the input pixels, we can iteratively update the image until its embedding matches the target vector.

Minimal code example:

```
# initializing a random image
image = torch.randn(1, 3, 256, 256, device=device,
requires_grad=True)

# setting the optimizer to control the image pixels
optimizer = torch.optim.Adam([image], lr=0.1)

# optimization loop
for i in range(iterations):
    optimizer.zero_grad()

    # forward pass
    embedding = facenet_model(image)

    # calculate loss
    loss = mse_loss(embedding, target_embedding)

    # backward pass (update image pixels)
    loss.backward()
    optimizer.step()
```

Until we have a moderate inversion attack method, we will use the same target image:



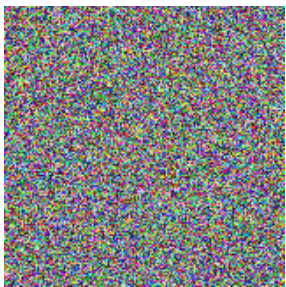
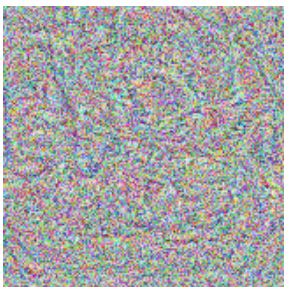
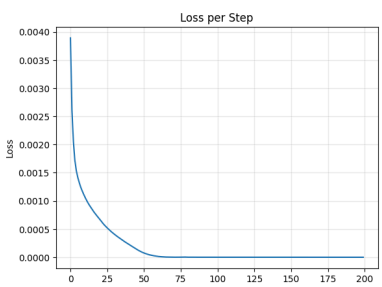
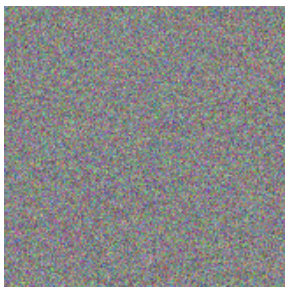
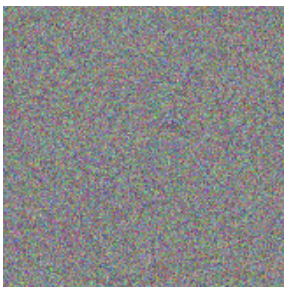
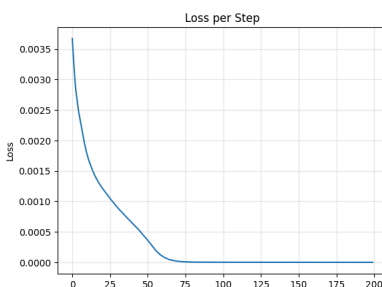
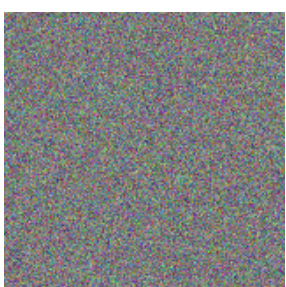
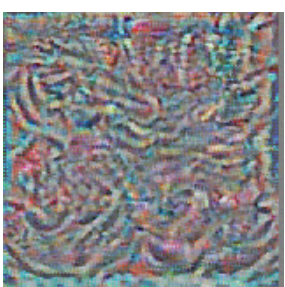
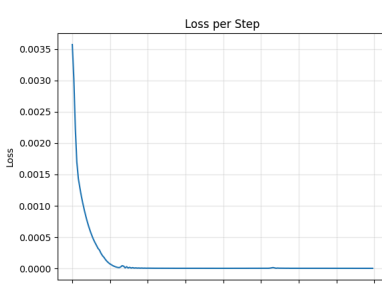
Target Image

Attempts 1-3 - experimenting with standard initializations

In the first attempts we tried various standard initializations of the reconstructed image:

1. Initializing our image with random noise from a uniform distribution.
2. Initializing our image with random noise from a gaussian distribution.
3. Initializing our image with a constant value + a very small gaussian noise.

These 3 attempts used the Adam optimizer and MSE loss, with 200 iterations on 0.01 learning rate.

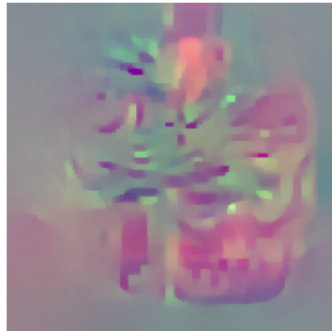
init method	start image	reconstructed image	loss graph
uniform			
gaussian			
const + gaussian			

In these attempts we can already see the problem in the classic method - the loss function isn't representative enough - we got it to reach 0, even though the reconstructed image is just a random noise, essentially creating an adversarial example.

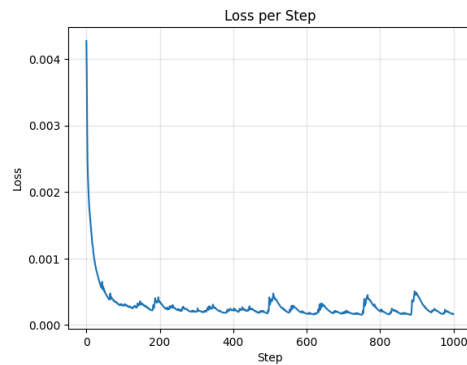
Attempt 4 - adding total variation to loss

We saw that the reconstructed images are very noisy. Total variation is used to make the image smoother by computing the difference between adjacent pixels in the image. After some hyperparameters tuning, we increased the iterations to 1000 and weighted the total loss as follows:

$$TotalLoss = MSELoss + 4 \cdot 10^{-7} \cdot TVLoss$$



Attempt 4 - Reconstructed Image



We can see that the reconstructed image is way less noisy and is smoother in the edges of the image than the center. While it fixed the noise, it failed to recover geometry because smoothness $\neq$ identity.

Attempt 5 - face initialization

Since we know that our target image is a face, we thought about starting with an image of a general face we found on the internet (not related to the CelebA-HQ dataset):



General Face Init

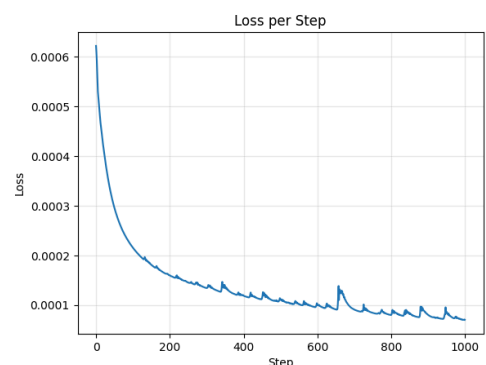
After running the optimization with total variation weight of $1 \cdot 10^{-7}$ we got the following results:



Attempt 5 - Reconstructed Image



Target Image



We can see that the reconstructed image finally starts to look like a face, and even resembles the target face image.

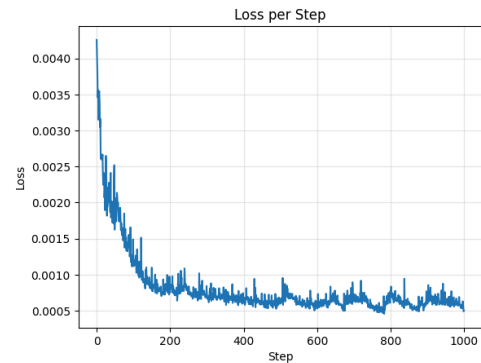
Attempt 6 - jittering the image

The next idea we had was slightly moving the image randomly in each step. This way, we make it harder for the optimization loop to fall into adversarial reconstructions, and hopefully instead, generate actual facial features.

We jittered the image 30 pixels max in each step using reflection padding. Also we used MSE and total variation loss, on a constant + small gaussian initialization.



Attempt 6 - Reconstructed Image



Compared to attempt 4 that used the same initialization and loss, we can see that the jittering helped and we can see an eye and maybe a nose in the image.

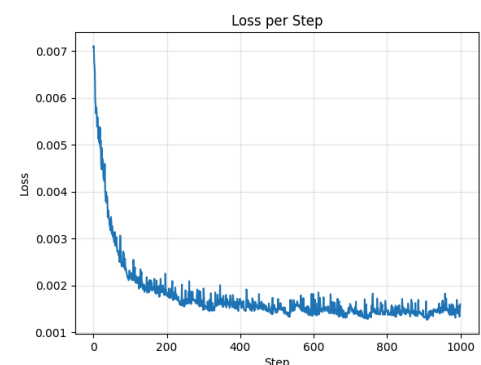
Attempt 7 - Adding Perceptual Distance to the Loss

The perceptual distance uses the vgg16 network up to layer 16. This sub-network takes an image and outputs an embedding vector, similar to FaceNet except it was trained on a different dataset (ImageNet). The logic behind using it is trying to reconstruct the target image as before, while also keeping the image similar to an actual face image. As an anchor, we used the general face image from attempt 5. The vgg16 sub-network creates embeddings for the general face image and the reconstructed image, and the loss part is the MSE distance between these embeddings:

$$TotalLoss = MSELoss + 4 \cdot 10^{-7} \cdot TVLoss + 2 \cdot 10^{-4} \cdot PerceptualLoss$$



Attempt 7 - Reconstructed Image



We see that the result image is too similar to the anchor image and not to the target image, and when we tried to decrease the perceptual loss weight, we got more and more weird artifacts that did not get us closer to the target.

Second Method - Generative Prior

Our second approach was led by attempt 5, when we initialized the image with a general face. We saw that the initialization of an actual face resulted in a better reconstruction than any other attempts, even complex ones that used jittering and perceptual loss. Part of why we tried to add the perceptual loss, was because we understood that in the optimization loop, the reconstructed image space should always look like an actual face. The search space should be reduced to only include valid faces.

The current SOTA method that does exactly that, is using a generative model that will generate our reconstructed image in each step. In the backpropagation, the gradients will flow through FaceNet into the image and then into the generative model, until the input that we feed it.

In our attempts, we used the model StyleGAN by NVIDIA, that was trained with the faces dataset FFHQ, and not the version that was trained on CelebA-HQ dataset so there won't be any data leakage.

While regular GANs get gaussian noise as input (Z), StyleGAN uses an intermediate space W which is copied and sent as instruction input in each of the 18 layers of the network. Intuitively, we can think of each layer as responsible of a specific face feature (e.g. eyes, skin texture), and the W as an instructor that says that this feature should look like an old person for example. However, we thought that W space may be insufficient because why not control every layer separately to increase the expressivity of the model. This is the $W+$ space and essentially we send different W vectors in each of the 18 layers, instead of copying the same vector 18 times. For initialization, we took the empirical mean of the latent space W . This is safer than starting with a random face because the center face should be closer to any other face on average.

```
def get_mean_w(self, n_samples=4096, seed=0):
    """ Get the average latent code repeated 18 times. """
    torch.manual_seed(seed)
    z = torch.randn(n_samples, self.G.z_dim, device=device)
    w = self.G.mapping(z, None) # Convert z space to w space
    w_avg = w.mean(0, keepdim=True)
    return w_avg # (1, 18, 512)
```

The Average Person (Mean Face)



Minimal code:

```
# initialize StyleGAN
generator = StyleGANGenerator('models/ffhq.pkl')

# initialize starting w+
latent_code = generator.get_mean_w()

# setting the optimizer to control the latent code
optimizer = optim.Adam([latent_code], lr=0.05)

# optimization loop
for i in range(iterations):
    optimizer.zero_grad()

    # generate the image with StyleGAN
    gen_image_1024 = generator(latent_code)

    # resize for FaceNet (160x160)
    gen_image_160 = F.interpolate(gen_image_1024, size=(160, 160),
mode='bilinear', align_corners=False)

    # pass through facenet
    embedding = facenet_model(gen_image_160)

    # calculate loss
    loss = mse_loss(embedding, target_embedding)

    # backward pass (update image pixels)
    loss.backward()
    optimizer.step()
```

Attempt 8 - Optimizing W

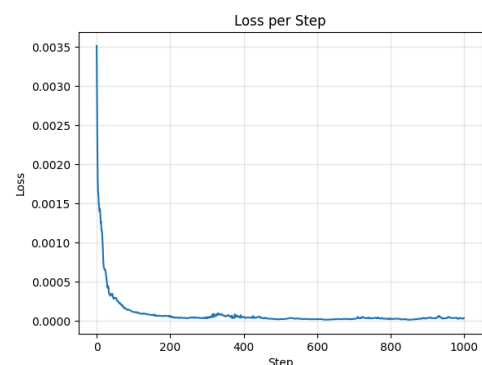
Our first attempt with the StyleGAN was actually optimizing w and not $w+$ to see whether it's sufficient. We optimized the 512 dimensional vector while copying it 18 times inside the loop before sending it to the generator.



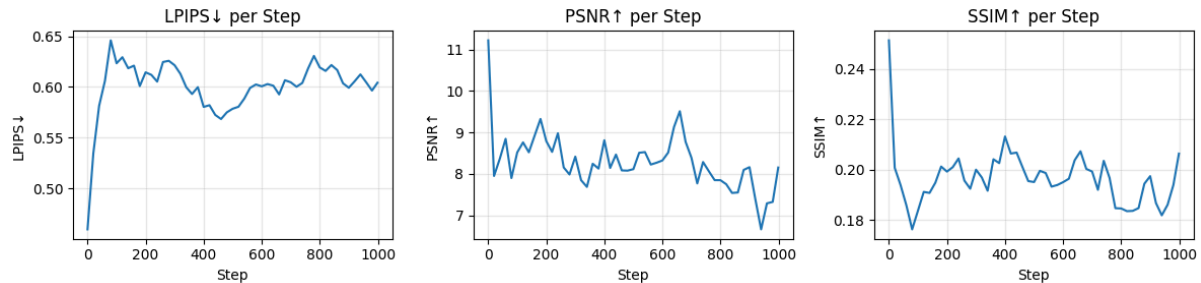
Target Image



Attempt 8 - Reconstructed Image

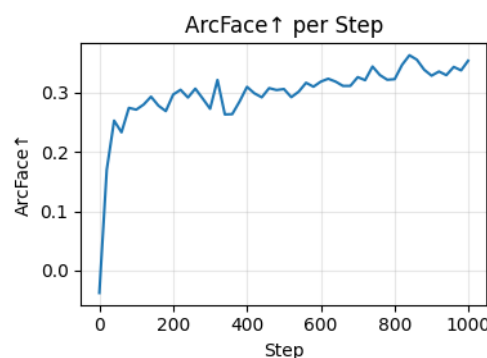


We can see a significant improvement in the GMI method. The reconstructed image looks a lot like a real face without any noise. It does contain weird artifacts, specifically here, the skin color got a bit green in some places, however the face looks very similar to the person in the target image. We can see the 3 evaluation metrics as we explained at the beginning:



We can see that their values are very suboptimal and unrepresentative of the reconstruction which we consider accurate. To add to that, the best metric values were received in step 0 where our image was the average face.

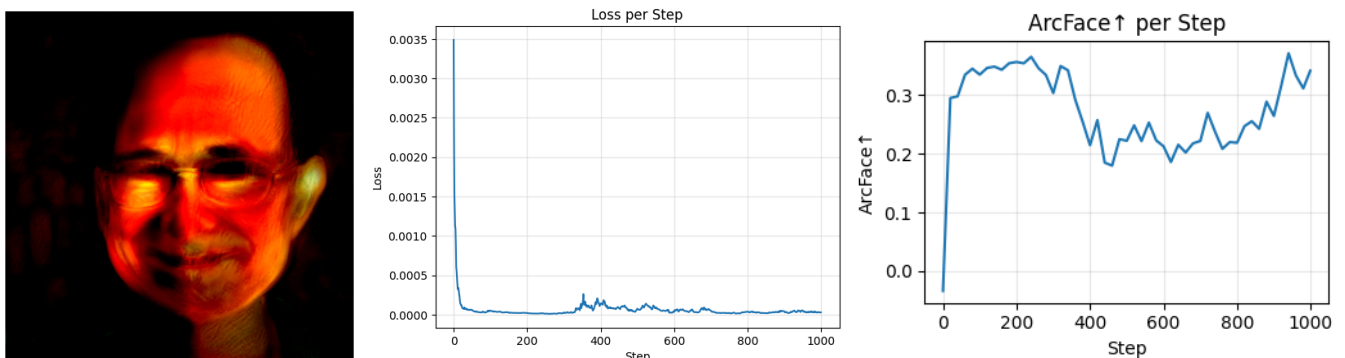
As a consequence, we decided to move into using the ArcFace model to measure perceptual distance between our reconstruction and the target image, so even though we computed the other metrics for the rest of the attempts as well, we will show only the ArcFace's.



The first GMI Attempt got a similarity score of 0.353 after 1000 iterations, which is a pretty good start.

Attempt 9 - Optimizing W+

Next, we wanted to check whether we can get more accurate results when optimizing w^+ which allows the model to have different "instruction" vectors for each face feature.



We can see that the reconstruction here is way less accurate than optimizing only w and the loss gets very small, so we can conclude that optimizing on $w+$ changes the face features too freely. This happens because $w+$ has 18 times more parameters than w , so by giving the optimizer that much more freedom from the start, it finds an adversarial shortcut (overfitting).

Attempt 10 - Coarse to Fine

We then thought about splitting the optimization loop into 2: first we will optimize w for a few iterations to get it in the area of the target image, then, we will optimize $w+$ with a lower learning rate that will hopefully get us more accurate face features.

We found out by trial and error that starting with 25 iterations on 0.05 lr on w , and then 100 iterations with 0.025 lr on $w+$ gives good results.

We will display the image we got after the first loop and the final reconstruction:



Attempt 10 - After w opt

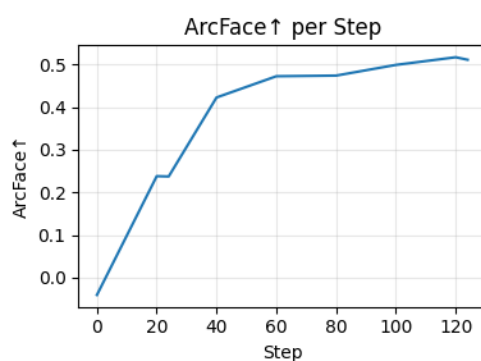
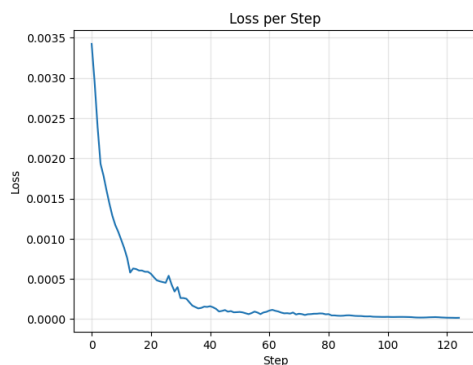


Attempt 10 - Reconstructed Image



Target Image

We can see significant improvements in this method. The second loop shifted the image into having more similar facial features such as face structure and mouth.



The final ArcFace metric value is 0.51 which is already a very good result.

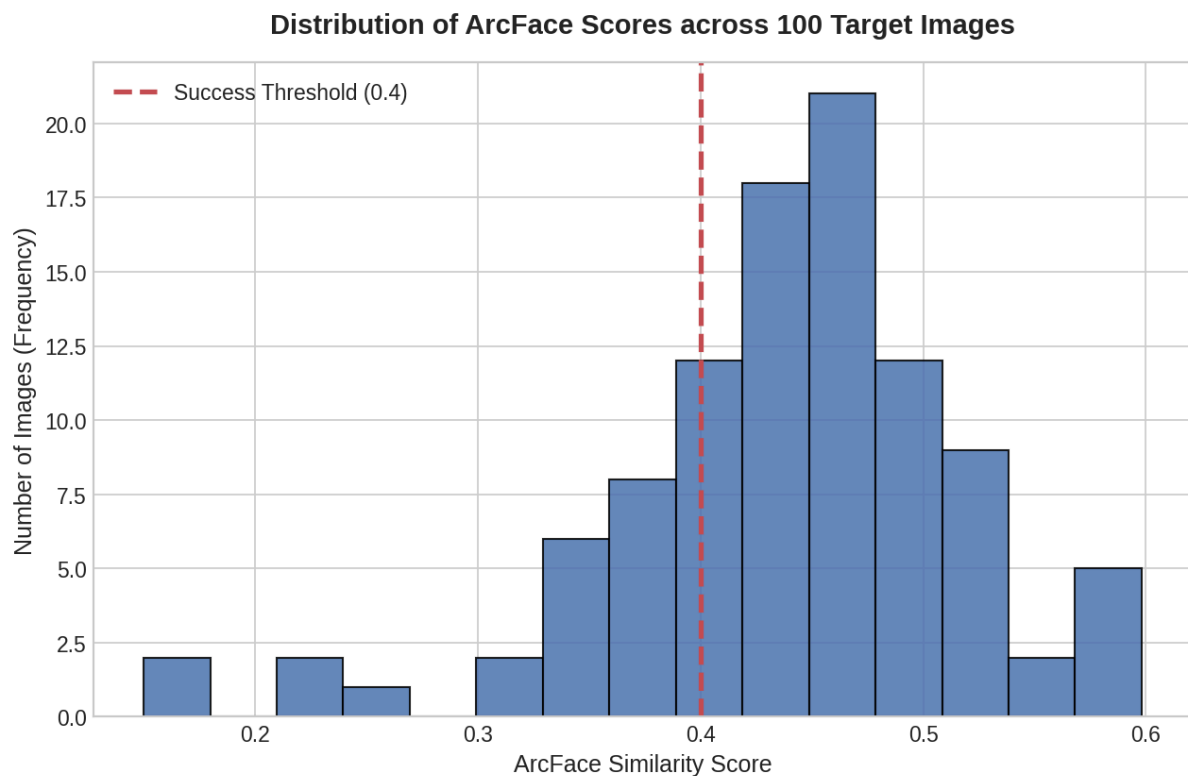
Exploring Different Seeds

We understand that this method can get us good results, but the problem is that it's very inconsistent with respect to the starting seed which slightly affects the starting w_{mean} image. This happens because the starting mean face is computed empirically (averaging 4096 w 's). For example, here are the results after the first loop, where the only thing that changed is the starting seed:



Final Results

We took the first 100 target images in the dataset. For each of them we tested various seeds and optimized w for 25 iterations and then $w+$ for additional 35 iterations. We stored the ArcFace metric value of the reconstructed image. As we saw in similar tasks, an ArcFace similarity metric of at least 0.4 is considered good, so we put this as our threshold. Here are the values per target image:

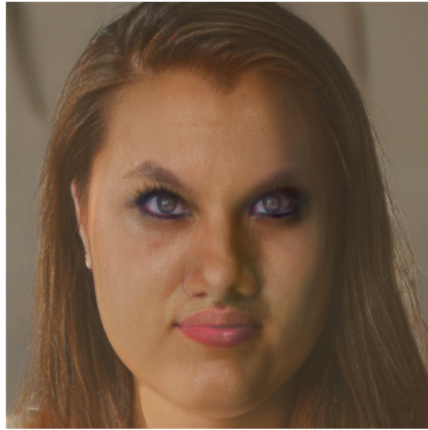


We succeeded at reconstructing 72 out of 100 target images. We will show several failed reconstructions and several good ones, based on the ArcFace score they got. The images on the left side are the targets and the images on the right are our reconstruction attempt.

- **Failed Results:**



Target Image 83



Reconstructed Image 83. ArcFace: 0.176



Target Image 0



Reconstructed Image 0. ArcFace: 0.215



Target Image 73



Reconstructed Image 73. ArcFace: 0.253

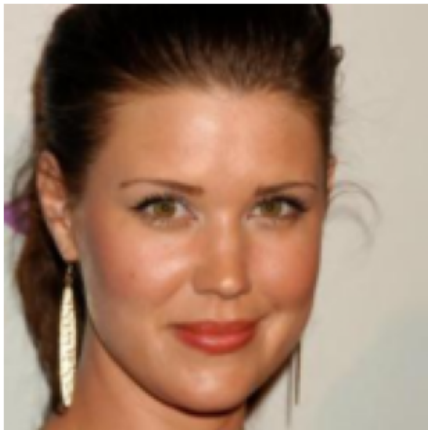
- **Good Results:**



Target Image 68



Reconstructed Image 68. ArcFace: 0.594



Target Image 66



Reconstructed Image 66. ArcFace: 0.583



Target Image 63



Reconstructed Image 63. ArcFace: 0.538

Conclusion

In this project, we demonstrated the vulnerability of facial image embeddings to Model Inversion Attacks. While our initial pixel-space reconstruction method yielded poor results, using a generative prior with StyleGAN allowed us to reconstruct highly recognizable faces that clearly resemble the target images.

An important finding in our Generative Model Inversion method was the need for a coarse-to-fine strategy. By initially optimizing W to locate the general facial structure, and then fine-tuning $W+$ to get the subtler facial features, we managed to create more accurate results, preventing the adversarial artifacts that were observed when we optimized $W+$ directly.

Finally, we achieved a 72% success rate across our CelebA-HQ test subset, surpassing an ArcFace similarity threshold of 0.4.

Future Work

Future study could focus on the sensitivity to the initial latent space sampling - the fact that the reconstruction result can vary significantly depending on the initial latent seed. Stabilizing the starting seed dependence would improve consistency and practicality of the attack.

How to Run

To run our inversion attack, follow the [instructions](#) in the github README file.